SVD

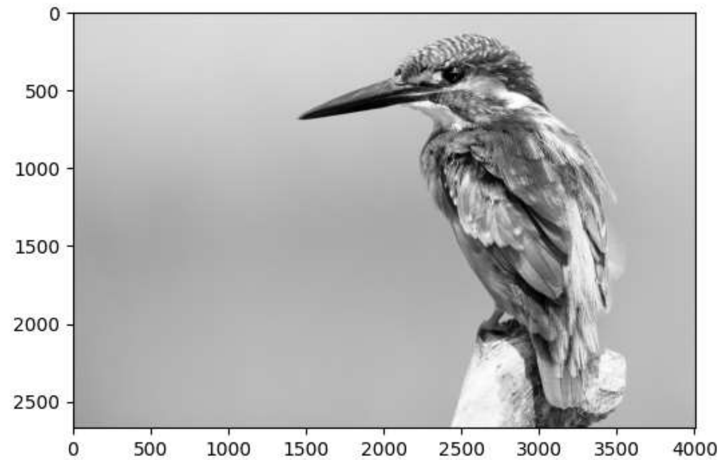
```
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image  #python imaging library
```

```
img = Image.open('bird.jpg')
plt.imshow(img)
img_gray=img
```


```
img.size
```

```
(4005, 2670)
```

```
img_gray = img.convert('LA')  #he  LA  mode utilizes both an alpha channel and "8-bit pixels in black
                              #alpha channel is a color component that represents the degree of transpa
plt.figure(figsize=(6, 6))
plt.imshow(img_gray);
```




```
img_gray.size
```

```
(4005, 2670)
```

```
img_mat = np.array(list(img_gray.getdata(band=0)), float)
img_mat.shape = (img_gray.size[1], img_gray.size[0])
img_mat = np.matrix(img_mat)
plt.figure(figsize=(9,6))
plt.imshow(img_mat, cmap='gray');
img_mat.shape
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-4009254045.py in <cell line: 0>()
----> 1 img_mat = np.array(list(img_gray.getdata(band=0)), float)
      2 img_mat.shape = (img_gray.size[1], img_gray.size[0])
      3 img_mat = np.matrix(img_mat)
      4 plt.figure(figsize=(9,6))
      5 plt.imshow(img_mat, cmap='gray');

NameError: name 'np' is not defined
```

```
img_mat
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-3960061114.py in <cell line: 0>()
----> 1 img_mat

NameError: name 'img_mat' is not defined
```

```
img_mat.shape
```

```
(2031, 3082)
```

```
U, sigma, V = np.linalg.svd(img_mat)
```

```
print("size of U is ", U.shape)
print("size of sigma is ",sigma.shape)
print("size of V is ", V.shape)
```

```
size of U is  (2031, 2031)
size of sigma is  (2031,)
size of V is  (3082, 3082)
```

```
print(sigma)
```

Show hidden output

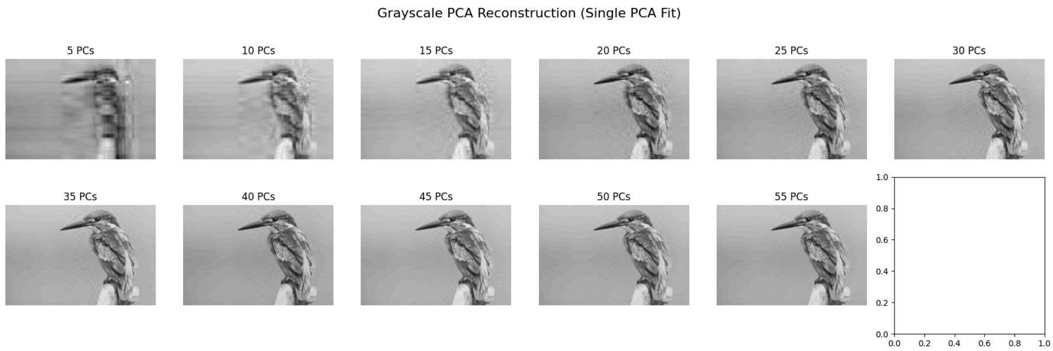
```
for i in range(5, 500, 10):
  a = np.matrix(U[:, 0:i]) * np.diag(sigma[:i]) * np.matrix(V[:i, :])
  plt.imshow(a, cmap='gray')
  title = "n = %s" % i
  plt.title(title)
  plt.show()
```

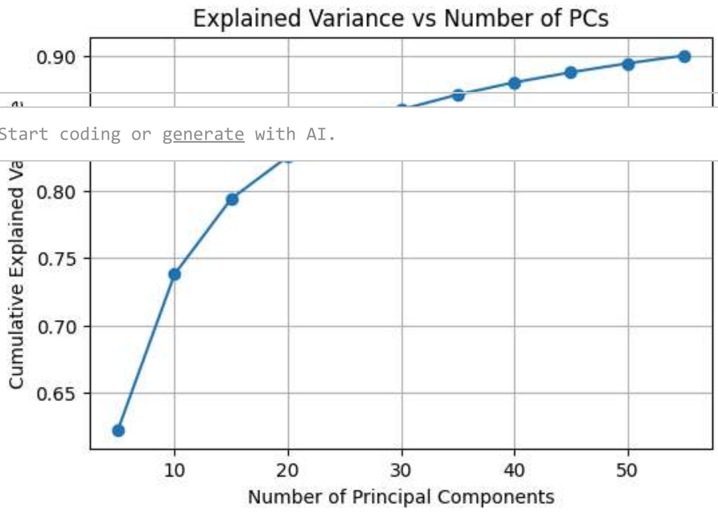
Show hidden output

```
a = np.matrix(U[:, 0:60]) * np.diag(sigma[:60]) * np.matrix(V[:60, :])
a.nbytes
```

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from sklearn.decomposition import PCA
```

```python
img = Image.open("bird.jpg").convert("L")
img = np.asarray(img, dtype=np.float32) / 255.0

h, w = img.shape

max_components = 55
pca = PCA(n_components=max_components)
scores = pca.fit_transform(img)


components_list = list(range(5, max_components + 1, 5))
explained_variance = []

fig, axes = plt.subplots(2, 6, figsize=(18, 6))
axes = axes.ravel()

for idx, k in enumerate(components_list):

    scores_k = scores[:, :k]
    components_k = pca.components_[:k, :]

    reconstructed = np.dot(scores_k, components_k)
    reconstructed += pca.mean_

    explained_variance.append(np.sum(pca.explained_variance_ratio_[:k]))

    axes[idx].imshow(reconstructed, cmap="gray")
    axes[idx].set_title(f"{k} PCs")
    axes[idx].axis("off")

plt.suptitle("Grayscale PCA Reconstruction (Single PCA Fit)", fontsize=16)
plt.tight_layout()
plt.show()

# Explained variance curve

plt.figure(figsize=(6, 4))
plt.plot(components_list, explained_variance, marker="o")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Explained Variance vs Number of PCs")
plt.grid(True)
plt.show()
```






```
Start coding or generate with AI.
```